Modeling Pips as a Constraint Satisfaction Problem - And Solving It

Kai Gowers

MATH4480 Spring 2026

1 "Pips" Overview

NYT Pips is a puzzle where you place dominoes on a weirdly-shaped grid. Each square ends up with a number (a "pip") from one side of a domino. Some groups of squares (regions) have constraints like:

- "sum of this region is 4"
- "all numbers in this region are equal"
- "sum of this region is less than 3"

The challenge is to place every domino exactly once, satisfying both the adjacency rules and regional constraints. My solver turns this into a constraint satisfaction problem and solves it automatically using the Z3 SMT solver.

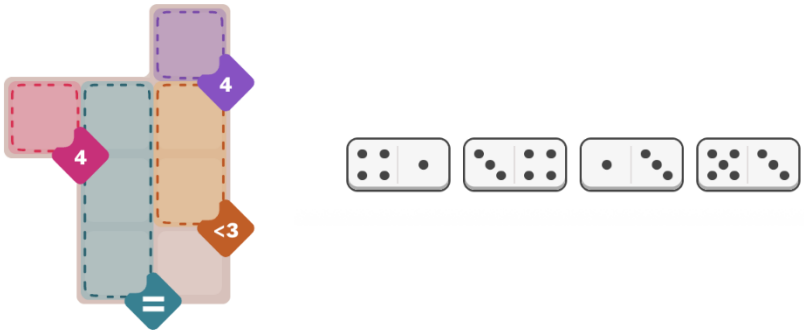


Figure 1: NYT Pips Easy Example

2 Introduction to the Z3 SMT Solver

The Z3 SMT Solver is a theorem prover developed by Microsoft Research that determines whether a collection of logical constraints can all be satisfied simultaneously. At its foundation is the Boolean satisfiability problem (SAT), which asks whether a propositional formula can be made true by assigning truth values to its variables. A propositional formula is a logical expression built from variables together with operations such as AND, OR, and NOT.

For example, consider the statement:

> "It is raining or I have an umbrella."

If we let

$$R = \text{“It is raining”}$$

$$U = \text{“I have an umbrella”}$$

then this statement can be written logically as

$$R \vee U$$

This is a simple example of a propositional formula. The SAT problem asks whether there exists some assignment of truth values to the variables that makes the formula true. For example, if

$$R = \text{False}, \qquad U = \text{True}$$

then the formula is satisfied.

SMT (Satisfiability Modulo Theories) extends SAT by allowing formulas to involve richer mathematical objects such as integers and real numbers rather than only Boolean variables. This allows SMT solvers to reason about mathematical constraints in addition to pure logical statements. For example, the formula

$$x + y = 10 \qquad \text{and} \qquad x > y$$

is not naturally a SAT problem because it involves integer arithmetic rather than only Boolean variables.

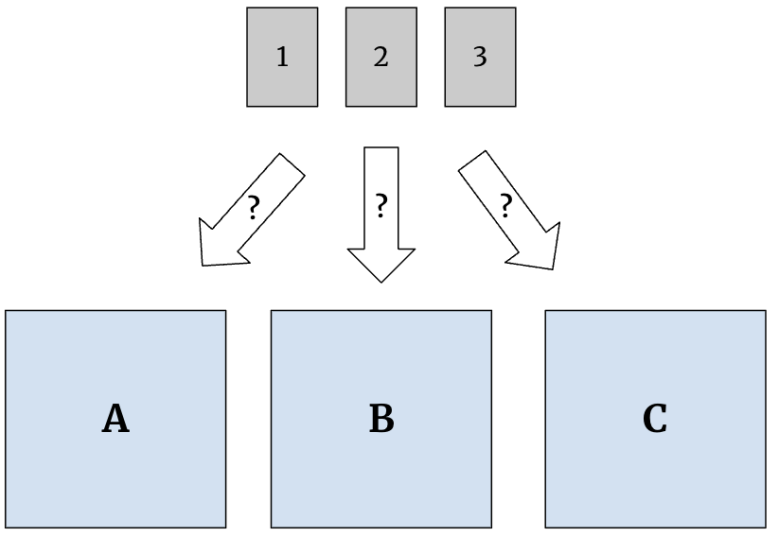
Computationally, SMT solvers combine logical search with mathematical verification. First, a SAT solver searches through possible combinations of logical constraints. Then, specialized mathematical procedures verify whether those choices are actually consistent within a given theory, such as arithmetic.

SMT is especially convenient for problems like Pips because the puzzle naturally contains both logical and combinatorial constraints. Rather than reducing every condition to pure Boolean logic, SMT allows the constraints to be expressed in a more direct mathematical form.

3 Simpler Example of Constraint Encoding to Build Intuition

Before encoding the full Pips puzzle, it is helpful to first look at a much smaller constraint satisfaction problem in order to build intuition for how we would encode Pips.

Suppose we want to place the numbers 1, 2, and 3 into three boxes labeled A, B, and C.



Each number must be used exactly once, and the following conditions must hold:

$$A + B = 5, \qquad A > C, \qquad A, B, C \in \{1, 2, 3\}$$

In Z3, this can be encoded as follows:

```
A = Int('A')
B = Int('B')
C = Int('C')

solver = Solver()

# Values must be between 1 and 3
solver.add(A >= 1, A <= 3)
solver.add(B >= 1, B <= 3)
solver.add(C >= 1, C <= 3)

# All values must be different
solver.add(Distinct(A, B, C))

# Additional constraints
solver.add(A + B == 5)
solver.add(A > C)
```

Here, the variables are represented symbolically rather than being assigned values immediately. The constraints describe the rules of the problem, and Z3 searches for values satisfying all of them simultaneously.

Z3 would find the one viable solution:

$$A = 3, \quad B = 2, \quad C = 1$$

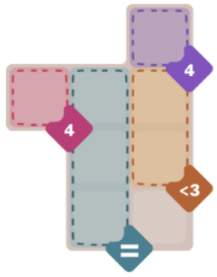
This example illustrates the core idea behind constraint encoding: instead of directly solving the problem by hand, we translate the rules into logical and mathematical constraints.

The Pips solver uses this same principle on a much larger scale. Rather than assigning numbers to three boxes, the project encodes domino placements and puzzle rules as logical constraints whose satisfiability corresponds to a valid puzzle solution.

4 Board Representation

4.1 Board

The puzzle board is represented as a 2D array in which valid cells hold temporary placeholders (e.g. numbers such as 0,1,2,...), while empty spaces contain the token `"-1"`.

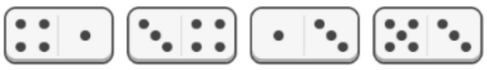



```
board = [
    [-1, -1, 0],
    [1, 2, 3],
    [-1, 4, 5],
    [-1, 6, 7]
]
```

4.2 Dominoes

Each domino is a pair (a, b) representing the pip values on its two halves. We index dominos as $d = 0, 1, \ldots, D-1$. Each domino may appear on any edge and in either orientation.

$$\text{dominos} = [(a_0, b_0), (a_1, b_1), \ldots].$$



```
dominos = [(4,1), (3,4), (1,3), (5,3)]
```

4.3 Region Constraints

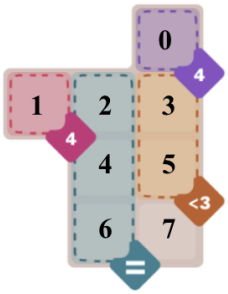
Regions are represented as tuples:

$$(\text{cells},\ \text{op},\ \text{target}).$$

The operator may be:

$$\text{sum_eq},\ \text{sum_lt},\ \text{sum_gt},\ \text{all_eq},\ \text{all_diff}.$$

which correspond to the possible constraints on a Pips Board, as seen in this next image.




```
regions = [
    ([1], "sum_eq", 4),
    ([2,4,6], "all_eq", None),
    ([0], "sum_eq", 4),
    ([3,5], "sum_lt", 3)
]
```

4.4 Edges

Two cells are adjacent if they are orthogonally next to each other (up, down, left, right). From this, we construct the edge list:

$$\text{edges} = \{(u, v) \mid u \text{ and } v \text{ are adjacent valid cells}\}.$$

```
edges = [(0, 3), (1, 2), (2, 3), (2, 4), (3, 5), (4, 5), (4, 6), (5, 7), (6, 7)]
```

5 Z3 Encoding

To solve the puzzle, we translate the board into a collection of logical variables and constraints evaluated by the Z3 solver. The encoding enforces three key ideas:

(a) every domino must be placed exactly once

(b) every cell must be covered by exactly one domino-side

(c) the number placed in each cell must agree with the pip value of the domino that covers it

5.1 Placement Variables

For each domino d, each edge e of the cell-adjacency graph, and each orientation $o \in \{0, 1\}$ (normal or reversed), we create a Boolean variable

$$\text{place}_{d,e,o}.$$

It is true exactly when domino d is placed on edge e with orientation o.

5.2 Cell Value Variables

For each cell c of the board we introduce an integer variable

$$v_c \in \{0, \ldots, 6\},$$

representing the pip value ultimately assigned to that cell.

5.3 Domino Placement Constraints

Each domino must be used exactly once. Since each possible placement corresponds to a Boolean variable, we enforce

$$\sum_{e,o} \text{place}_{d,e,o} = 1 \qquad \text{for every domino } d.$$

This ensures that Z3 chooses one, and only one, placement-orientation pair for each domino.

5.4 Cell Coverage Constraints

For each cell c, we precompute the set of all possible placements that could cover it. Exactly one of these placements must be chosen:

$$\sum_{(d,e,o) \text{ covers } c} \text{place}_{d,e,o} = 1.$$

This prevents overlap and guarantees that every cell is filled by exactly one domino-side.

5.5 Cell Value Consistency

If a placement (d, e, o) would assign pip value x to a cell c, then whenever that placement is selected, the value of the cell must match x. This is expressed with implications:

$$\text{place}_{d,e,o} \;\Rightarrow\; v_c = x.$$

Because exactly one placement covering c is true, this pins down v_c to the pip value contributed by the chosen domino.

5.6 Region Constraints

Each region R on the board specifies a numerical rule on the cell values within it. These rules translate naturally into arithmetic constraints over the v_c. Examples include:

$$\sum_{c \in R} v_c = T, \qquad v_{c_1} = v_{c_2} \text{ for all } c_1, c_2 \in R,$$

or other constraints. These formulas capture the semantics of the puzzle's region labels.

6 Solving

Once all constraints have been added, we ask Z3 to determine whether the system is satisfiable and, if so, to produce a model. Internally, Z3 searches through the space of possible variable assignments while checking whether the constraints remain consistent.

In the context of Pips, the model specifies which domino placements are true, allowing us to extract the orientation and position of each domino on the board.

```
placements (that are true) = [(0, 0, 0), (1, 1, 1), (2, 5, 1), (3, 8, 1)]

# (d, e, o) --> domino d is placed on edge e with orientation o
```

7 Visualization

We render the final board on a 2D grid.

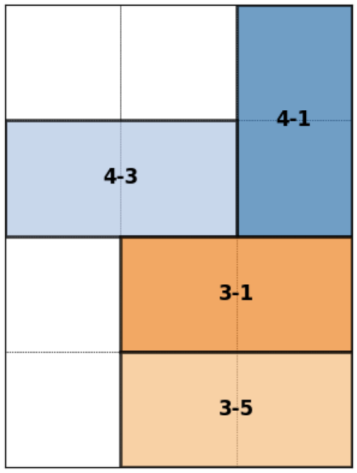


Figure 2: NYT Pips Easy Example Solution

8 Conclusion

By translating the puzzle into a constraint satisfaction model and leveraging Z3's reasoning capabilities, we obtain a general-purpose solver that works for arbitrary Pips boards.